PHASE 1A: EVENT TICKETING PLATFORM

Sections 3, 4 & 5 — Implementation Reference

Feature Order • Backend Architecture • Frontend Architecture • Full Project Structure

SECTION 3

Feature Implementation Order

Every feature across all 21 days — with prerequisites, acceptance criteria, and test requirements

This section maps every feature to its build day, estimated effort, required prerequisites, definition of done, and test requirements. Use it as a daily tracking sheet — cross off each row when the acceptance criteria are met.

Days 1–7: Backend Foundation Layer

Day	Feature	Hrs	Prerequisites	Acceptance Criteria	Tests Required
1	Project setup + DB schema	5 hrs	None	App starts, Flyway runs, all 9 tables created in PostgreSQL	No tests yet (infrastructure only)
1	All JPA entities	1.5 hrs	DB schema	All entities compile, relationships correct, @Version on Booking	JPA validation annotations present
2	Auth service (Phase 0 reuse)	2 hrs	Entities	JWT login working, ORGANIZER role present in token	UserServiceTest: 3+ tests
2	Event Service (TDD)	3.5 hrs	Auth, Entities	All 5 CRUD endpoints work, authorization enforced per role	EventServiceTest: 7+ tests
3	Category + Venue CRUD	2 hrs	Entities	GET/POST for categories and venues returning correct data	3+ tests each service
3	Event Search	1.5 hrs	Event Service	Filter by q, category, city — returns only PUBLISHED future events	SearchTest: 3+ tests
4	Next.js initialization	1.5 hrs	None (frontend)	App runs on port 3000, home page renders without errors	Manual test
4	Home Page + EventCard	3 hrs	Event API working	Real events displayed in grid, category filter triggers new API call	Manual test
5	InventoryService	3 hrs	Entities, Redis	Availability tracked in Redis, 100-thread concurrency test passes	InventoryTest: 4+ tests
5	Redis Caching (Events)	1.5 hrs	Redis, EventService	Second fetch for same event hits Redis cache (SQL logs show 0 queries)	CacheTest: 2+ tests

Day	Feature	Hrs	Prerequisites	Acceptance Criteria	Tests Required
5	RabbitMQ Config	**1 hr**	RabbitMQ running	All queues + DLQs declared and visible in Management UI on startup	No tests (config only)
6	N+1 Query Fixes	**2 hrs**	Event + Booking repos	SQL logs show ≤ 3 queries for any list fetch of 10 events	N+1 findings documented in README
6	Integration Test Setup	**2 hrs**	Testcontainers	Testcontainers starts PostgreSQL, Redis, and RabbitMQ successfully	1 passing integration test
7	Docker Compose (full)	**1.5 hrs**	All services	docker-compose up starts all 5 services, docker-compose ps shows all healthy	docker-compose ps shows no unhealthy containers
7	Code review + cleanup	**2 hrs**	All Week 1 code	No Clean Code violations (method length, naming, magic strings)	—

Days 8–14: Core Business Logic

Day	Feature	Hrs	Prerequisites	Acceptance Criteria	Tests Required
8	State Machine Config	**2.5 hrs**	Spring SM library	All transitions defined, guards wired, state machine factory configured	StateMachineTests: 5+
8	BookingService.reserveTickets()	**2.5 hrs**	State Machine, Redis	Seat locked in Redis, Booking created in RESERVED state atomically	Integration test
8	Reservation expiry job	**1 hr**	State Machine	RESERVED bookings automatically expire after 5 minutes	@Scheduled test
9	PaymentService + Stripe SDK	**3 hrs**	Stripe account, Booking	Checkout URL returned from API, Payment entity created with PENDING status	PaymentServiceTest: 4+
9	Stripe Webhook handler	**2 hrs**	PaymentService	Payment success →	WebhookTest: 3+

Day	Feature	Hrs	Prerequisites	Acceptance Criteria	Tests Required
				CONFIRMED state; payment failure → RELEASED state	
10	RabbitMQ publishers	**2 hrs**	RabbitMQ Config	Messages published to correct queues, verified in Management UI	Publisher verified via Management UI
10	Notification listeners	**2 hrs**	Publishers	Booking confirmation email delivered to Mailhog inbox	ListenerIntegrationTest
10	QR code generation	**1 hr**	ZXing library	QR code Base64 string saved on ticket, renders correctly as image	QRCodeTest: 2+
11	DistributedLockService	**2.5 hrs**	Redis	Lua script lock/release working; concurrent test: 100 users → exactly 50 succeed	ConcurrentBookingTest
12	PricingEngine	**2.5 hrs**	TicketTier entity	All 4 pricing rules work correctly, edge cases covered	PricingEngineTest: 8+ tests, 100% branch coverage
12	WaitlistService	**1.5 hrs**	Redis, Notification	Users join waitlist, notified via RabbitMQ when a seat is released	WaitlistTest: 3+
12	RefundService	**2 hrs**	Payment, State Machine	3-tier refund calculation correct, Stripe refund API called with right amount	RefundServiceTest: 5+
13	Event Detail page (frontend)	**2 hrs**	Event API	Tiers displayed with prices and availability, "Add to Cart" works	Manual test
13	Cart + Countdown Timer	**2 hrs**	Booking API	5-min countdown	Manual test

Day	Feature	Hrs	Prerequisites	Acceptance Criteria	Tests Required
				visible, reaching zero shows expiry modal	
13	Confirmation page	**1.5 hrs**	Booking API, QR	QR codes rendered as images, booking reference visible and copyable	Manual test
14	Full booking integration test	**2 hrs**	All booking features	End-to-end test passes: reserve → checkout → confirm → QR generated	1 comprehensive integration test
14	System Design practice	**1 hr**	—	"Design Ticketmaster" architecture diagram complete	Diagram screenshot saved

Days 15–21: Frontend, Testing & Deployment

Day	Feature	Hrs	Prerequisites	Acceptance Criteria	Tests Required
15	User Dashboard	**2.5 hrs**	Auth, Booking API	Bookings listed, upcoming events shown, stats row correct	Manual test
15	Organizer Dashboard	**2.5 hrs**	Event API, Analytics	Create-event form works end-to-end, revenue displayed in analytics card	Manual test
15	Checkout page	**1.5 hrs**	Payment API	Form pre-fills from user profile, redirects to Stripe hosted page	Manual test
16	Test coverage push	**5.5 hrs**	All services written	JaCoCo ≥ 80% overall; 100% branch coverage on critical paths	15+ new tests
17	k6 load testing	**3.5 hrs**	App deployed locally	P95 < 500ms, 0 double-bookings, error rate < 1%	k6 output documented in README
17	Connection pool tuning	**1.5 hrs**	k6 results	No HikariCP pool exhaustion under 1,000 virtual users	—

Day	Feature	Hrs	Prerequisites	Acceptance Criteria	Tests Required
18	GitHub Actions CI/CD	**2.5 hrs**	GitHub repo	Tests run on every PR, Docker image built on push to main	CI green badge on repo
18	Railway deployment	**2 hrs**	Dockerfile, CI	Live URL responding, /actuator/health returns {"status":"UP"}	URL verified manually
19	Swagger UI	**1 hr**	All controllers	All endpoints documented at /swagger-ui.html with request/response schemas	Manual UI verification
19	README completion	**3 hrs**	All features done	Architecture diagram, screenshots, quick-start, env variables table present	—
20	Bug bash	**2.5 hrs**	Full feature walkthrough	Zero critical bugs in happy path flows against live Railway URL	Manual E2E
20	Frontend error/loading states	**1 hr**	Frontend pages	No blank screens on API failure, empty states with CTAs present	Manual test
21	Portfolio preparation	**1.5 hrs**	All complete	LinkedIn updated, 3-min Loom demo video recorded and linked	—
21	Phase 1B planning	**1 hr**	Phase 1A done	Gaps honestly identified, next project direction decided	—

SECTION 4

Backend Architecture Detail

Package structure, database schema, API endpoints, and the booking flow sequence

4.1 Complete Backend Package Structure

The backend follows a **domain-module architecture** — each business domain (event, booking, payment, notification, inventory, pricing, user) is a self-contained package with its own controller, service, repository, model, and DTO layers. Shared infrastructure lives in the `common/` package.

Root: `src/main/java/com/ticketing/`

event/ — Event, Category & Venue domain

Controllers

- `EventController.java` — Full CRUD plus publish and close-sales operations. Annotated with `@PreAuthorize` for organizer-only mutations.
- `CategoryController.java` — Category CRUD (admin-managed).
- `VenueController.java` — Venue CRUD (admin-managed).

Services

- `EventService.java` — Core event lifecycle: create, read, update, delete, publish.
- `EventSearchService.java` — Full-text and filter-based search with pagination.
- `CategoryService.java` — Category management.
- `VenueService.java` — Venue management.

Repositories

- `EventRepository.java` — Custom JPQL queries including `findByIdWithDetails` (JOIN FETCH) and search with optional filters.
- `CategoryRepository.java`, `VenueRepository.java` — Standard JPA repositories.

Models

- `Event.java` — `@Entity`; carries `@Version Long version` for optimistic locking.
- `EventStatus.java` — Enum: `DRAFT → PUBLISHED → SALES_OPEN → SALES_CLOSED → COMPLETED → ARCHIVED`.
- `Category.java`, `Venue.java` — Standard `@Entity` classes.

DTOs

- `CreateEventRequest.java`, `UpdateEventRequest.java` — Input validation with `@NotBlank`, `@Future`, `@Min`.
- `EventResponse.java` — Full event details for single-event calls.
- `EventSummaryResponse.java` — Lightweight response for list views (fewer fields, faster serialization).
- `EventFilterRequest.java` — Query parameters for the search endpoint.

booking/ — Booking lifecycle & State Machine

Controllers

- `BookingController.java` — Endpoints: reserve, proceed to checkout, get booking, cancel, list user bookings.

Services

- `BookingService.java` — Core booking logic: seat reservation, lock acquisition, state transitions, expiry handling.
- `BookingStateMachineService.java` — Wraps Spring State Machine. Restores state from DB per request, sends the event, persists new state back (stateless-per-request pattern).
- `ReservationExpirationJob.java` — `@Scheduled(fixedDelay = 30000)` job. Runs every 30 seconds; finds bookings where `state=RESERVED AND expires_at < now` and fires `TIMER_EXPIRED` on each.

Repositories

- `BookingRepository.java`, `TicketRepository.java`.

Models

- `Booking.java` — `@Entity`; carries `@Version Long version` as a second safety net against concurrent writes. Stores `expires_at` for the 5-minute reservation window.
- `BookingState.java` — Enum: `AVAILABLE, RESERVED, PAYMENT_PENDING, CONFIRMED, ATTENDED, EXPIRED, RELEASED, PAYMENT_FAILED, REFUND_REQUESTED, REFUND_APPROVED, REFUND_DENIED`.
- `BookingEvent.java` — Enum (state machine triggers): `ADD_TO_CART, PROCEED_TO_CHECKOUT, PAYMENT_SUCCESS, PAYMENT_FAILURE, TIMER_EXPIRED, CHECK_IN, REQUEST_REFUND, APPROVE_REFUND, DENY_REFUND`.
- `Ticket.java` — `@Entity`; one row per physical ticket. Stores `qr_code` as Base64 text.

State Machine (booking/statemachine/)

- `BookingStateMachineConfig.java` — Annotated `@EnableStateMachineFactory`. Wires all states, transitions, guards, and actions.

- **Guards —** `guards/SeatsAvailableGuard.java` (blocks ADD_TO_CART when tier is sold out), `guards/BookingOwnedByUserGuard.java` (blocks cancel/refund if wrong user).
- **Actions —** `actions/StartReservationTimerAction.java` (sets `expires_at`), `ConfirmBookingAction.java` (updates payment status), `ReleaseSeatsAction.java` (increments inventory + notifies waitlist), `GenerateQRCodeAction.java` (creates Base64 QR for each ticket).

DTOs

- `ReserveTicketRequest.java`, `BookingResponse.java`, `CheckoutSessionResponse.java`.

payment/ — Stripe payments & refunds

Controllers

- `PaymentController.java` — Create checkout session (`POST /api/payments/checkout/{bookingId}`), get payment status.
- `StripeWebhookController.java` — `POST /api/webhooks/stripe`. Verifies Stripe signature, checks idempotency, routes to handlers.

Services

- `PaymentService.java` — Builds Stripe `SessionCreateParams` from booking tickets, calls `Session.create()`, persists Payment entity.
- `RefundService.java` — Calculates refund amount via `PricingEngine`, calls Stripe `Refund.create()`, updates booking state to `REFUND_APPROVED`.

Repositories & Models

- `PaymentRepository.java`, `RefundRepository.java`.
- `Payment.java` — Stores `stripe_session_id`, `stripe_payment_intent_id`, amount, currency, status.
- `PaymentStatus.java` — Enum: `PENDING, COMPLETED, FAILED, REFUNDED`.
- `Refund.java` — Stores `stripe_refund_id`, refund amount, reason, status, processed timestamp.
- `RefundStatus.java` — Enum: `PENDING, APPROVED, DENIED`.

notification/ — Async messaging & email

Listeners (consume RabbitMQ queues)

- `BookingNotificationListener.java` — `@RabbitListener` on `booking.confirmation.queue`. Fetches booking, generates PDF, sends confirmation email. Manual ACK with `channel.basicNack()` on failure → routes to DLQ.
- `EmailNotificationListener.java` — `@RabbitListener` on `email.notification.queue`. Delegates to `EmailService.sendEmail()`.
- `TicketGenerationListener.java` — `@RabbitListener` on `ticket.generation.queue`. Calls `ticketService.generateQrCodes(ticketIds)`.

Services & Publishers

- `EmailService.java` — `JavaMailSender` wrapper. Sends HTML emails with inline Base64 QR code images.
- `BookingEventPublisher.java` — `RabbitTemplate` wrapper with three publish methods, auto JSON-serialized via `Jackson2JsonMessageConverter`.

Message POJOs (DTOs)

- `BookingConfirmedEvent.java` — Payload for booking confirmation queue. Contains bookingId, userId, email, ticketIds, event info.
- `EmailNotificationEvent.java`, `TicketGenerationEvent.java`, `WaitlistAvailableEvent.java`.

inventory/ • pricing/ • user/

inventory/ — Redis-backed availability

- `InventoryService.java` — Manages ticket availability counts in Redis. Key methods: `getAvailableCount()`, `reserveSeat()`, `releaseSeat()`, `commitReservation()`.
- `WaitlistService.java` — Redis Sorted Set waitlist. Key methods: `joinWaitlist()`, `notifyNextInWaitlist()`.
- `TicketTierRepository.java` — Shared repository for tier availability queries.

pricing/ — All pricing and refund rules

- `PricingEngine.java` — Stateless service containing all 4 pricing rules (dynamic surge, early bird, group discount, VIP tier base) and the 3-tier refund calculation. No external dependencies — pure business logic, 100% testable.

user/ — Authentication and user management

- `UserController.java` — `GET /api/users/me` (current user profile), `GET /api/users/me/bookings` (user booking history).
- `UserService.java`, `UserDetailsServiceImpl.java` — Spring Security user loading from the database.
- `UserRepository.java`, `User.java`, `Role.java` (enum: `USER, ORGANIZER, ADMIN`).

common/ — Shared infrastructure (config, security, exceptions, utilities)

config/

- `SecurityConfig.java` — JWT filter chain, CORS configuration, role-based access rules.
- `RabbitMQConfig.java` — Declares exchanges (`booking.exchange` topic, `notification.exchange` direct), queues, DLQs, bindings, and registers `Jackson2JsonMessageConverter`.
- `RedisConfig.java` — Configures `RedisTemplate<String, Object>` with Jackson serialization, and `RedisCacheManager` with per-cache TTL overrides.
- `StripeConfig.java` — `@PostConstruct` sets `Stripe.apiKey` from environment variable.
- `SwaggerConfig.java` — OpenAPI info (title, version, description, contact).

exception/

- `GlobalExceptionHandler.java` — `@ControllerAdvice` mapping all custom exceptions to structured `ApiResponse` error responses with appropriate HTTP status codes.
- Custom exceptions: `EventNotFoundException`, `BookingNotFoundException`, `InsufficientInventoryException`, `InvalidStateTransitionException`, `LockAcquisitionException`, `RefundNotEligibleException`.

security/

- `JwtService.java` — Issues and validates JWT tokens (HS256, configurable expiry).
- `JwtAuthenticationFilter.java` — Extends `OncePerRequestFilter`; extracts JWT from Authorization header, validates, and populates the Spring Security context.

util/

- `QRCodeGeneratorUtil.java` — ZXing wrapper. Encodes a signed JSON payload as `BarcodeFormat.QR_CODE` at 200×200px and returns Base64.
- `DateUtils.java` — Helper methods for the refund policy date calculations.
- `ApiResponse.java` — Standard response wrapper used by all controllers: `{ status, data, error }`.

4.2 Database Schema (ERD)

The schema contains **9 tables** across 3 logical domains. All primary keys use `BIGSERIAL` (auto-incrementing 64-bit integers). `created_at` and `updated_at` timestamps are present on all mutable entities. Foreign key relationships enforce referential integrity at the database level.

> **Optimistic Locking:** The version column (BIGINT, default 0) on both events and bookings is managed by Hibernate @Version. On every update, Hibernate increments this value and adds WHERE version = {old_value} to the UPDATE statement. If another transaction modified the row first, the update affects 0 rows and Hibernate throws ObjectOptimisticLockingFailureException — the second safety net against concurrent booking corruption.

users

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
email	VARCHAR(255)	NOT NULL UNIQUE
password_hash	VARCHAR(255)	NOT NULL — BCrypt hash, never stored plain
first_name	VARCHAR(100)	Nullable
last_name	VARCHAR(100)	Nullable
role	VARCHAR(20)	NOT NULL DEFAULT 'USER' — values: USER, ORGANIZER, ADMIN
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMP	Nullable — set on update

categories

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL UNIQUE — e.g. Music, Sports, Comedy, Theater, Festival
description	TEXT	Nullable
icon_url	VARCHAR(500)	Nullable — emoji or image URL for UI display

venues

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
address	VARCHAR(500)	NOT NULL

Column	Type / Size	Constraints & Notes
city	VARCHAR(100)	NOT NULL — indexed for city-based event search
country	VARCHAR(100)	NOT NULL DEFAULT 'US'
total_capacity	INT	NOT NULL
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()

events

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
title	VARCHAR(255)	NOT NULL
description	TEXT	Nullable
organizer_id	BIGINT	NOT NULL — FK → users(id), indexed
category_id	BIGINT	NOT NULL — FK → categories(id)
venue_id	BIGINT	NOT NULL — FK → venues(id)
cover_image_url	VARCHAR(500)	Nullable — URL to banner image
start_date	TIMESTAMP	NOT NULL — indexed for date-range queries
end_date	TIMESTAMP	NOT NULL
sales_open_date	TIMESTAMP	Nullable — when ticket sales begin
sales_close_date	TIMESTAMP	Nullable — when ticket sales end
status	VARCHAR(30)	NOT NULL DEFAULT 'DRAFT'
dynamic_pricing_enabled	BOOLEAN	NOT NULL DEFAULT FALSE
waitlist_enabled	BOOLEAN	NOT NULL DEFAULT FALSE
version	BIGINT	NOT NULL DEFAULT 0 — @Version optimistic locking
created_at / updated_at	TIMESTAMP	Standard audit columns

ticket_tiers

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
event_id	BIGINT	NOT NULL — FK → events(id) ON DELETE CASCADE
tier_name	VARCHAR(50)	NOT NULL — e.g. VIP, GOLD, STANDARD
description	VARCHAR(500)	Nullable

Column	Type / Size	Constraints & Notes
base_price	DECIMAL(10,2)	NOT NULL — starting price before any rules applied
total_capacity	INT	NOT NULL — fixed total seats for this tier
available_count	INT	NOT NULL — decremented on reserve, incremented on release
max_per_booking	INT	NOT NULL DEFAULT 10 — max tickets per single booking
UNIQUE	—	Composite unique constraint on (event_id, tier_name)

bookings

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
user_id	BIGINT	NOT NULL — FK → users(id), indexed
event_id	BIGINT	NOT NULL — FK → events(id), indexed
booking_date	TIMESTAMP	NOT NULL DEFAULT NOW()
state	VARCHAR(30)	NOT NULL DEFAULT 'RESERVED' — indexed for expiry job queries
total_amount	DECIMAL(10,2)	NOT NULL — calculated at reservation time
version	BIGINT	NOT NULL DEFAULT 0 — CRITICAL: @Version optimistic locking
expires_at	TIMESTAMP	Nullable — RESERVED state expires after 5 min. Partial index: WHERE state='RESERVED'
stripe_session_id	VARCHAR(255)	Nullable — set when checkout session created
created_at / updated_at	TIMESTAMP	Standard audit columns

tickets

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
booking_id	BIGINT	NOT NULL — FK → bookings(id) ON DELETE CASCADE
tier_id	BIGINT	NOT NULL — FK → ticket_tiers(id)
seat_number	VARCHAR(20)	Nullable — assigned seat if applicable
qr_code	TEXT	Nullable until payment confirmed — Base64-encoded QR image
check_in_status	BOOLEAN	NOT NULL DEFAULT FALSE

Column	Type / Size	Constraints & Notes
check_in_time	TIMESTAMP	Nullable — set when attendee checks in at door
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()

payments

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
booking_id	BIGINT	NOT NULL UNIQUE — FK → bookings(id) — one payment per booking
stripe_session_id	VARCHAR(255)	Set when Stripe Checkout session is created
stripe_payment_intent_id	VARCHAR(255)	Set when payment is confirmed — used for refunds
amount	DECIMAL(10,2)	NOT NULL
currency	VARCHAR(3)	NOT NULL DEFAULT 'USD'
status	VARCHAR(20)	NOT NULL — values: PENDING, COMPLETED, FAILED, REFUNDED
payment_method	VARCHAR(50)	Nullable — e.g. card, apple_pay
created_at / updated_at	TIMESTAMP	Standard audit columns

refunds

Column	Type / Size	Constraints & Notes
id	BIGSERIAL	PRIMARY KEY
payment_id	BIGINT	NOT NULL — FK → payments(id)
stripe_refund_id	VARCHAR(255)	Nullable until processed — set by Stripe
amount	DECIMAL(10,2)	NOT NULL — calculated by PricingEngine refund rules
reason	VARCHAR(500)	Nullable — customer-supplied reason
status	VARCHAR(20)	NOT NULL DEFAULT 'PENDING' — values: PENDING, APPROVED, DENIED
requested_at	TIMESTAMP	NOT NULL DEFAULT NOW()
processed_at	TIMESTAMP	Nullable — set when Stripe processes the refund

Performance Indexes

The following indexes are created in V7__create_indexes.sql to support the most frequent query patterns:

Index Name	On Column(s)	Supports
idx_events_start_date	events(start_date)	Future event queries and date-range filters
idx_events_status	events(status)	All queries filtering by PUBLISHED, SALES_OPEN, etc.
idx_events_organizer_id	events(organizer_id)	Organizer dashboard — list my events
idx_bookings_user_id	bookings(user_id)	User dashboard — list my bookings
idx_bookings_event_id	bookings(event_id)	Organizer dashboard — attendee list per event
idx_bookings_state	bookings(state)	Expiry job — find all RESERVED bookings
idx_bookings_expires_at	bookings(expires_at) WHERE state='RESERVED'	Partial index — expiry job only scans active reservations (much smaller)
idx_ticket_tiers_event_tier	ticket_tiers(event_id, tier_name) UNIQUE	Prevents duplicate tier names per event + fast tier lookups

4.3 Booking Flow — Sequence Diagram (Happy Path)

The following describes the complete flow from a user selecting a seat through receiving their confirmation. Actors involved: **User**, **Frontend**, **Booking API**, **InventoryService**, **Redis**, **Stripe**, and **RabbitMQ**.

Phase 1: Reservation (POST /api/bookings/reserve)

1. **User → Frontend:** User selects a ticket tier and quantity on the Event Detail page.
2. **Frontend → Booking API:** `POST /api/bookings/reserve` with `{ eventId, tierId, quantity }` and JWT token.
3. **Booking API → InventoryService:** Calls `getAvailableCount(tierId)`.
4. **InventoryService → Redis:** Reads `inventory:tier:{tierId}:available`. Returns current count (e.g. 50).
5. **Booking API → Redis:** Acquires distributed lock: `SET seat:lock:tier:{tierId}:user:{userId} {uuid} NX EX 300`. If lock already held → returns HTTP 409.
6. **Booking API → InventoryService:** Double-checks availability while holding the lock (TOCTOU protection). Calls `reserveSeat(tierId, quantity)` to atomically decrement Redis count.
7. **Booking API → Database:** Creates `Booking` entity with `state=RESERVED`, `expires_at = now + 5 minutes`. Creates one `Ticket` row per quantity.
8. **Booking API → Redis:** Stores cart data: `HSET cart:{userId} bookingId, tierId, quantity, eventId`.

9. **Booking API → Frontend:** Returns `BookingResponse` containing `bookingId` and `expiresAt`.
10. **Frontend → User:** Navigates to `/cart`. Countdown timer starts from `expiresAt - now`.

Phase 2: Checkout (POST /api/bookings/{id}/checkout)

11. **User → Frontend:** Clicks "Proceed to Checkout".
12. **Frontend → Booking API:** `POST /api/bookings/{bookingId}/checkout`.
13. **Booking API → State Machine:** Sends `PROCEED_TO_CHECKOUT` event. Booking transitions from `RESERVED → PAYMENT_PENDING`.
14. **Booking API → Stripe:** Calls `Session.create(SessionCreateParams)` with line items built from `booking.getTickets()`, success URL, cancel URL, 30-minute expiry, and metadata `{ bookingId, userId }`.
15. **Booking API → Database:** Saves `stripe_session_id` on the booking and creates a `Payment` entity with `status=PENDING`.
16. **Booking API → Frontend:** Returns `CheckoutSessionResponse { stripeUrl }`.
17. **Frontend → User:** `window.location.href = stripeUrl`. User is redirected to Stripe's hosted payment page.

Phase 3: Payment Confirmation (Stripe Webhook → confirmed)

18. **Stripe → Booking API:** `POST /api/webhooks/stripe` with event type `checkout.session.completed` and a `Stripe-Signature` header.
19. **Booking API:** Verifies signature with `Webhook.constructEvent(payload, sigHeader, webhookSecret)`. Checks idempotency — if this event ID was already processed, returns `200 OK` immediately.
20. **Booking API → State Machine:** Extracts `bookingId` from session metadata. Sends `PAYMENT_SUCCESS` event. Booking transitions `PAYMENT_PENDING → CONFIRMED`.
21. **GenerateQRCodeAction fires:** ZXing encodes a signed JSON payload for each ticket, stores Base64 string in `tickets.qr_code`.
22. **Booking API → Database:** Updates `Payment` entity: `status=COMPLETED`, `stripe_payment_intent_id` set.
23. **Booking API → Redis:** Releases distributed lock: Lua script atomically checks ownership and deletes.
24. **Booking API → RabbitMQ:** Publishes `BookingConfirmedEvent` to `booking.exchange` with routing key `booking.confirmed`. This is async — the API responds immediately without waiting for email delivery.
25. **Frontend → User:** Stripe redirects to `success_url`. Confirmation page loads, displays QR codes from `tickets[].qrCode`.

Phase 4: Async Post-Confirmation (RabbitMQ)

26. **RabbitMQ → BookingNotificationListener:** Delivers `BookingConfirmedEvent` from `booking.confirmation.queue`.
27. **BookingNotificationListener → EmailService:** Sends HTML confirmation email with ticket details and inline QR code images.
28. **On failure:** Listener calls `channel.basicNack(deliveryTag, false, false)`. Message routes to `booking.confirmation.dlq` for manual inspection — user is not affected.

4.4 API Endpoints Summary

Events

Method	Endpoint	Description	Auth Required
POST	/api/events	Create new event in DRAFT status	ORGANIZER role
GET	/api/events	List published events (paginated + filtered)	Public
GET	/api/events/{id}	Get full event detail with venue, tiers, organizer	Public
PUT	/api/events/{id}	Update event (organizer only, DRAFT/PUBLISHED)	ORGANIZER (owner)
DELETE	/api/events/{id}	Delete event (DRAFT only)	ORGANIZER (owner)
POST	/api/events/{id}/publish	Transition DRAFT → PUBLISHED	ORGANIZER (owner)
GET	/api/search/events	Full-text + filter search with pagination	Public

Bookings

Method	Endpoint	Description	Auth Required
POST	/api/bookings/reserve	Reserve seats (acquires Redis lock)	Authenticated
POST	/api/bookings/{id}/checkout	Create Stripe checkout session	Authenticated (owner)
GET	/api/bookings/{id}	Get booking details and ticket info	Authenticated (owner)
DELETE	/api/bookings/{id}	Cancel booking + trigger refund flow	Authenticated (owner)
GET	/api/users/me/bookings	List all bookings for current user	Authenticated

Payments & Webhooks

Method	Endpoint	Description	Auth Required
GET	/api/payments/{bookingId}	Get payment status for a booking	Authenticated (owner)
POST	/api/webhooks/stripe	Stripe event handler (signature-verified)	Stripe signature only

Categories, Venues & Users

Method	Endpoint	Description	Auth Required
GET	/api/categories	List all categories	Public
POST	/api/categories	Create category	ADMIN role
GET	/api/venues	List all venues	Public
POST	/api/venues	Create venue	ADMIN role
GET	/api/users/me	Get current user profile	Authenticated
POST	/api/auth/register	Register new account	Public
POST	/api/auth/login	Authenticate, receive JWT	Public

SECTION 5

Frontend Architecture Detail

Next.js 14 App Router structure, component hierarchy, state management strategy, and API integration

5.1 Frontend Technology Stack

All packages are installed into the frontend/ directory.

Package	Version / Source	Purpose
next	14 (App Router)	Full-stack React framework — server components, file-based routing, API route handlers
typescript	—	Static typing across all components, hooks, and API types
tailwindcss	—	Utility-first CSS — all styling, responsive breakpoints, animate-pulse skeletons
@tanstack/react-query	—	All server state: events, bookings. Handles caching, refetching, loading states automatically
axios	—	HTTP client. Configured with base URL + request/response interceptors
zustand	—	Client-only state: auth user, JWT token, cart reservation timer
react-hook-form	—	All form state — registration, login, create event, checkout
zod	—	Schema validation for all forms. Integrated with react-hook-form resolver
@stripe/stripe-js	—	Stripe.js browser SDK — loads Stripe Elements
@stripe/react-stripe-js	—	React wrapper for Stripe Elements (used on checkout page)
lucide-react	—	Icon library — calendar, map-pin, ticket, user, shopping-cart icons
date-fns	—	Date formatting and manipulation (countdown timer, event date display)
recharts	—	Charts on Organizer Dashboard — ticket sales over time line chart

5.2 Page Routing Structure (Next.js 14 App Router)

All pages live under `src/app/` and follow the Next.js 14 App Router file conventions. Every folder with a `page.tsx` becomes a route. `loading.tsx` files provide automatic streaming skeletons while data fetches. `layout.tsx` files wrap child routes with shared UI.

Folder	File	Page Name	Description
app/		**Root directory**	
	layout.tsx	**RootLayout**	Wraps every page: Navbar, Footer, ReactQueryProvider, AuthProvider
	page.tsx	**Home**	Hero, category filter pills, events grid — public, no auth required
	loading.tsx	**Home skeleton**	Pulse skeleton for events grid while data loads
events/[id]/	page.tsx	**Event Detail**	Tier selector, quantity picker, live price preview, Add to Cart — public
events/[id]/	loading.tsx	**Event Detail skeleton**	
cart/	page.tsx	**Cart**	Item list, 5-min countdown timer, order total, Proceed to Checkout — auth required
cart/	loading.tsx	**Cart skeleton**	
checkout/	page.tsx	**Checkout**	Order summary, attendee info form, Complete Purchase → Stripe redirect
bookings/[id]/confirmation/	page.tsx	**Confirmation**	Success animation, booking reference, QR code cards, Download PDF — auth required
dashboard/	page.tsx	**User Dashboard**	Profile, stats, upcoming bookings, booking history — auth guard
dashboard/	loading.tsx	**Dashboard skeleton**	
organizer/events/	page.tsx	**Organizer Dashboard**	My events table, analytics card, Create Event modal — ORGANIZER role guard
organizer/events/new/	page.tsx	**Create Event**	Multi-step form: Basic → Schedule → Tiers → Review — ORGANIZER role guard
auth/login/	page.tsx	**Login**	Email + password form, JWT stored in Zustand auth store
auth/register/	page.tsx	**Register**	Name + email + password form, role selection (USER or ORGANIZER)
api/auth/[...nextauth]/	route.ts	**Auth handler**	Custom JWT handling or NextAuth route handler

5.3 Component Hierarchy

All components live under `src/components/` and are organized by domain. Atomic UI primitives live in `ui/`. Context providers live in `providers/`.

layout/ — Site-wide shell

Component	Responsibilities
Navbar.tsx	Logo, search bar, auth buttons (login/register or user avatar), cart icon with reservation count badge
Footer.tsx	Links, company info
PageContainer.tsx	Max-width wrapper with consistent horizontal padding — used inside every page

events/ — Event browsing and detail

Component	Responsibilities
EventCard.tsx	Cover image (with gradient fallback), title, date badge, location icon + city, "From $XX" price, "Book Now" CTA button. Used in home grid and dashboard.
EventGrid.tsx	Responsive CSS grid container rendering EventCards. Adjusts columns from 1 (mobile) to 3 (desktop).
CategoryFilter.tsx	Horizontal scrolling row of pill buttons. Clicking a pill sets the active category filter and triggers a new React Query fetch.
EventSearch.tsx	Search text input + city dropdown + date range picker. Controlled inputs that update query params.
TicketTierCard.tsx	Tier name, price (with crossed-out original if discount active), available count badge, Select button — disabled with "Sold Out" badge when count = 0.
TicketQuantitySelector.tsx	+ and – buttons with min=1 and max=10 validation. Displays current quantity, live price preview updates as value changes.

booking/ — Cart, checkout and confirmations

Component	Responsibilities
CartItem.tsx	Event thumbnail, event name, event date, tier name, quantity, price per ticket, line total, Remove button.
CountdownTimer.tsx	MM:SS display counting down. Turns red when < 60 seconds remain. Fires onExpire callback at zero, which shows the expiry modal.
PriceBreakdown.tsx	Subtotal row, discount badge (if applicable), grand total. Updates reactively as cart contents change.
QRCodeDisplay.tsx	Renders the Base64 QR code string as an <img> element. Used on the confirmation page for each ticket.
BookingStatusBadge.tsx	Color-coded pill: CONFIRMED (green), RESERVED (blue), EXPIRED (gray), PAYMENT_FAILED (red), REFUNDED (orange).

organizer/ — Organizer tools

Component	Responsibilities
CreateEventForm.tsx	Multi-step form with 4 steps (BasicInfo → Schedule → Tiers → Review). Each step validates before proceeding. Submits on Step 4 "Publish".
EventAnalyticsCard.tsx	recharts LineChart showing tickets sold over time. Data fetched from the organizer analytics endpoint.
AttendeeTable.tsx	Paginated data table of attendees for a specific event. Columns: name, email, tier, check-in status. Export to CSV button.

ui/ — Design system atoms

Component	Responsibilities
Button.tsx	Variants: primary (filled navy), secondary (outlined), danger (red), ghost (transparent). Accepts loading state (shows spinner).
Input.tsx	Label, input field, optional helper text below, error state styling (red border + error message).
Modal.tsx	Portal-based overlay. Renders children in a centered dialog with backdrop blur. Closes on Escape key or backdrop click.
Badge.tsx	Small colored pill for status labels. Color determined by status value prop.
Skeleton.tsx	Pulse animation placeholder. Accepts width and height props for different shapes.
Toast.tsx	Success (green) and error (red) notification that appears at top-right and auto-dismisses after 4 seconds.
EmptyState.tsx	Illustration placeholder + heading + message + optional CTA button. Used when lists have no items.

providers/ — React context providers

Component	Responsibilities
AuthProvider.tsx	Zustand-backed auth context. Exposes: user, token, login(), logout(), isOrganizer(). Token persisted to localStorage.
CartProvider.tsx	Zustand-backed cart context. Exposes: currentBookingId, reservationExpiresAt, setReservation(), clearCart(). Timer state drives the CountdownTimer.
QueryProvider.tsx	Wraps children in React Query's QueryClientProvider. Configures default staleTime (5 min) and retry behavior.

5.4 State Management Strategy

State is divided into three clear categories, each handled by the right tool:

Tool	Manages	Why this tool
React Query (TanStack)	Server data — events list, event detail, booking history, payment status	Handles cache TTL (5 min for events), background refetching, loading/error states, and cache invalidation after mutations automatically
Zustand	Client state — auth user object, JWT token, cart reservation (bookingId + expiresAt)	Minimal boilerplate, no Provider wrapping needed, supports localStorage persistence for JWT
React Hook Form + Zod	All form state — login, register, create event, checkout attendee info	Uncontrolled form performance (no re-render per keystroke), Zod schema is the single source of truth for both validation and TypeScript types

Zustand Store Contracts

authStore.ts — persisted to `localStorage`:

- `user: User | null` — the logged-in user object (id, name, email, role).
- `token: string | null` — JWT token, auto-attached to every API request via Axios interceptor.
- `login(email, password): Promise<void>` — calls `POST /api/auth/login`, stores user and token.
- `logout(): void` — clears user and token, redirects to `/auth/login`.
- `isOrganizer(): boolean` — helper returning `user?.role === 'ORGANIZER'`.

cartStore.ts — in-memory only, not persisted:

- `currentBookingId: string | null` — the active RESERVED booking ID.
- `reservationExpiresAt: Date | null` — the exact expiry timestamp from the API response.
- `setReservation(bookingId, expiresAt)` — called by `useReserveTickets` on mutation success.
- `clearCart()` — called on expiry, cancellation, or successful confirmation.

5.5 API Integration Pattern

All API communication goes through a single configured `axios` instance defined in `src/lib/api.ts`. A custom React Query hook wraps each API operation, so components never call `axios` directly.

Axios Instance Configuration (src/lib/api.ts)

- **Base URL:** Created with `axios.create({ baseURL: process.env.NEXT_PUBLIC_API_URL })`. The `NEXT_PUBLIC_API_URL` env var points to the Spring Boot backend (`http://localhost:8080` in local dev, Railway URL in production).
- **Request interceptor:** Before every request, reads `useAuthStore.getState().token` and appends `Authorization: Bearer {token}` to the headers. If no token, request goes unauthenticated.
- **Response interceptor (success):** Unwraps the `ApiResponse` wrapper — extracts `response.data.data` so hooks receive the payload directly, not the envelope.
- **Response interceptor (error):** If HTTP 401 → calls `useAuthStore.getState().logout()` to clear credentials and redirect to login. For all other errors → rejects with `response.data.error` message string (or "An error occurred" as fallback).

React Query Hooks (src/hooks/)

Every API operation has a dedicated hook. This keeps components clean and makes the API contract explicit:

Hook	Type	Behavior
useEvents(filters)	useQuery	Fetches event list. queryKey includes filters so each filter combination has its own cache entry. staleTime: 5 minutes.
useEvent(id)	useQuery	Fetches single event with full details. staleTime: 5 minutes.
useReserveTickets()	useMutation	Posts to /api/bookings/reserve. onSuccess: calls cartStore.setReservation() and invalidates events queries to refresh availability counts.
useCreateCheckoutSession()	useMutation	Posts to /api/bookings/{id}/checkout. onSuccess: redirects to Stripe URL via window.location.href.
useBooking(id)	useQuery	Fetches booking + ticket details (including QR codes) for confirmation page.
useUserBookings()	useQuery	Fetches current user's full booking history for the dashboard.
useLogin()	useMutation	Posts to /api/auth/login. onSuccess: calls authStore.login() to persist token.
useCreateEvent()	useMutation	Posts to /api/events. onSuccess: invalidates organizer event list cache.

5.6 Route Protection & Auth Guard Pattern

Protected pages are guarded using a `withAuth` higher-order component or an inline auth check at the top of the `page.tsx`. The check reads `useAuthStore.getState().token` and calls `redirect('/auth/login')` (Next.js server-side redirect) if absent.

- **Public pages:** `/ (Home), /events/[id], /auth/login, /auth/register` — no auth check.
- **Auth-required pages:** `/cart, /checkout, /bookings/[id]/confirmation, /dashboard` — redirects to login if no token.
- **ORGANIZER-only pages:** `/organizer/events, /organizer/events/new` — checks `isOrganizer()`; shows 403 page if USER role.

SECTION

Full Project Structure

Complete annotated file tree — backend (Spring Boot) + frontend (Next.js) + infrastructure

This is the definitive reference for the full `ticketing-platform` repository layout. Every file listed here must exist by Day 21 for Phase 1A completion.

Backend — Spring Boot (Java 17)

Root: `ticketing-platform/` (project root)

Root & Config Files

- `pom.xml` — *Maven build file — all 20+ dependencies declared here*
- `docker-compose.yml` — *Full local dev environment: app, postgres, redis, rabbitmq, mailhog, redis-commander*
- `Dockerfile` — *Multi-stage build: eclipse-temurin:17-jdk-alpine builder + 17-jre-alpine runtime*
- `.gitignore` — *Excludes target/, .env, application-local.yml, *.class*
- `README.md` — *Complete documentation: diagrams, screenshots, quick start, env vars, load test results*
- `ticketing-platform.postman_collection.json` — *Postman collection for all API endpoints*

src/main/java/com/ticketing/

- `TicketingApplication.java` — *Spring Boot entry point (@SpringBootApplication)*

event/ module

- `event/controller/`
  - `EventController.java` — *CRUD + publish + close-sales (5 endpoints)*
  - `CategoryController.java` — *Category CRUD*
  - `VenueController.java` — *Venue CRUD*
- `event/service/`
  - `EventService.java` — *Core event lifecycle*
  - `EventSearchService.java` — *Full-text + filter search*
  - `CategoryService.java`
  - `VenueService.java`
- `event/repository/`
  - `EventRepository.java` — *Custom @Query methods + @EntityGraph*
  - `CategoryRepository.java`
  - `VenueRepository.java`
- `event/model/`

- Event.java — *@Entity + @Version Long version*
- EventStatus.java — *Enum: DRAFT→PUBLISHED→SALES_OPEN→SALES_CLOSED→COMPLETED→ARCHIVED*
- Category.java — *@Entity*
- Venue.java — *@Entity*
- event/dto/
  - CreateEventRequest.java — *Input DTO with @NotBlank, @Future, @Min validation*
  - UpdateEventRequest.java
  - EventResponse.java — *Full event detail response*
  - EventSummaryResponse.java — *Lightweight list-view response*
  - EventFilterRequest.java — *Search query parameters*

booking/ module

- booking/controller/
  - BookingController.java — *reserve, checkout, get, cancel, user bookings*
- booking/service/
  - BookingService.java — *Core booking logic — seat reservation, lock, state transitions*
  - BookingStateMachineService.java — *Spring SM wrapper — restore state, send event, persist new state*
  - ReservationExpirationJob.java — *@Scheduled(fixedDelay=30000) — expires stale RESERVED bookings*
- booking/repository/
  - BookingRepository.java
  - TicketRepository.java
- booking/model/
  - Booking.java — *@Entity + @Version Long version + expires_at*
  - BookingState.java — *Enum: 11 states*
  - BookingEvent.java — *Enum: 9 state machine triggers*
  - Ticket.java — *@Entity — one row per physical ticket*
- booking/statemachine/
  - BookingStateMachineConfig.java — *@EnableStateMachineFactory — all transitions, guards, actions*
  - guards/
    - SeatsAvailableGuard.java — *Blocks ADD_TO_CART when tier is sold out*
    - BookingOwnedByUserGuard.java — *Blocks cancel/refund if requester != booking owner*
  - actions/
    - StartReservationTimerAction.java — *Sets expires_at = now + 5 min*
    - ConfirmBookingAction.java — *Updates payment status on PAYMENT_SUCCESS*
    - ReleaseSeatsAction.java — *Increments inventory + notifies waitlist*
    - GenerateQRCodeAction.java — *ZXing QR generation for each ticket*
- booking/dto/
  - ReserveTicketRequest.java
  - BookingResponse.java
  - CheckoutSessionResponse.java

payment/ module

- payment/controller/
  - PaymentController.java — *Create checkout session, get payment status*
  - StripeWebhookController.java — *POST /api/webhooks/stripe — signature-verified*
- payment/service/
  - PaymentService.java — *Stripe SessionCreate + idempotency handling*
  - RefundService.java — *PricingEngine + Stripe Refund.create()*
- payment/repository/
  - PaymentRepository.java
  - RefundRepository.java
- payment/model/
  - Payment.java — *@Entity — stripe_session_id, stripe_payment_intent_id*
  - PaymentStatus.java — *Enum: PENDING, COMPLETED, FAILED, REFUNDED*
  - Refund.java — *@Entity — stripe_refund_id, amount, status*
  - RefundStatus.java — *Enum: PENDING, APPROVED, DENIED*

notification/ module

- notification/listener/
  - BookingNotificationListener.java — *@RabbitListener — booking.confirmation.queue*
  - EmailNotificationListener.java — *@RabbitListener — email.notification.queue*
  - TicketGenerationListener.java — *@RabbitListener — ticket.generation.queue*
- notification/service/
  - EmailService.java — *JavaMailSender wrapper — HTML email with inline QR*
- notification/publisher/
  - BookingEventPublisher.java — *RabbitTemplate wrapper — 3 publish methods*
- notification/dto/
  - BookingConfirmedEvent.java — *Message payload — bookingId, userId, email, ticketIds*
  - EmailNotificationEvent.java
  - TicketGenerationEvent.java
  - WaitlistAvailableEvent.java

inventory/ + pricing/ + user/ modules

- inventory/service/
  - InventoryService.java — *Redis-backed availability — getAvailableCount, reserveSeat, releaseSeat*
  - WaitlistService.java — *Redis Sorted Set — joinWaitlist, notifyNextInWaitlist*
- inventory/repository/
  - TicketTierRepository.java
- pricing/service/
  - PricingEngine.java — *All 4 pricing rules + 3-tier refund calculation — 100% branch coverage required*

- user/controller/
  - UserController.java — *GET /api/users/me and /me/bookings*
- user/service/
  - UserService.java
  - UserDetailsServiceImpl.java — *Spring Security UserDetailsService implementation*
- user/repository/
  - UserRepository.java
- user/model/
  - User.java — *@Entity*
  - Role.java — *Enum: USER, ORGANIZER, ADMIN*

common/ module (shared infrastructure)

- common/config/
  - SecurityConfig.java — *JWT filter chain, CORS, role-based access rules*
  - RabbitMQConfig.java — *Exchanges, queues, DLQs, Jackson converter*
  - RedisConfig.java — *RedisTemplate, RedisCacheManager, TTL configuration*
  - StripeConfig.java — *@PostConstruct sets Stripe.apiKey*
  - SwaggerConfig.java — *OpenAPI info*
- common/exception/
  - GlobalExceptionHandler.java — *@ControllerAdvice — maps all exceptions to ApiResponse errors*
  - EventNotFoundException.java
  - BookingNotFoundException.java
  - InsufficientInventoryException.java
  - InvalidStateTransitionException.java
  - LockAcquisitionException.java
  - RefundNotEligibleException.java
- common/security/
  - JwtService.java — *Issue + validate HS256 tokens*
  - JwtAuthenticationFilter.java — *OncePerRequestFilter — JWT → SecurityContext*
- common/util/
  - QRCodeGeneratorUtil.java — *ZXing wrapper — signed JSON → Base64 QR*
  - DateUtils.java — *Refund policy date helpers*
  - ApiResponse.java — *Standard wrapper: { status, data, error }*

src/main/resources/

- application.yml — *All config reading from env vars (no hardcoded values)*
- application-local.yml — *Hardcoded local dev values (in .gitignore — never committed)*
- application-test.yml — *Test profile — uses Testcontainers dynamic ports*
- db/migration/ — *9 Flyway SQL files*
  - V1__create_users_table.sql
  - V2__create_venues_and_categories.sql

- V3__create_events_table.sql
- V4__create_ticket_tiers.sql
- V5__create_bookings_and_tickets.sql
- V6__create_payments_and_refunds.sql
- V7__create_indexes.sql — *All 8 performance indexes*
- V8__add_event_features.sql — *waitlist_enabled, dynamic_pricing_enabled columns*
- V9__seed_data.sql — *5 categories + 3 venues for dev/test*

src/test/java/com/ticketing/

- BaseIntegrationTest.java — *Abstract base — @SpringBootTest + @Testcontainers, shared containers for PostgreSQL, Redis, RabbitMQ*
- EventServiceTest.java — *Unit tests — 8+ tests, Mockito mocks*
- EventControllerTest.java — *@WebMvcTest — all endpoints, auth tests*
- EventIntegrationTest.java — *Testcontainers — full CRUD + search*
- BookingStateMachineTest.java — *5+ state transition tests*
- ConcurrentBookingTest.java — *100 threads → exactly 50 succeed, 50 fail*
- PaymentServiceTest.java — *Mockito — Stripe SDK mocked*
- PaymentWebhookTest.java — *Valid/invalid signature, idempotency*
- PricingEngineTest.java — *8+ tests — 100% branch coverage required*
- InventoryServiceTest.java — *Testcontainers Redis — concurrency test*
- NotificationListenerIntegrationTest.java — *Testcontainers RabbitMQ — Awaitility*
- WaitlistIntegrationTest.java
- RefundServiceTest.java — *5+ tests — all refund tiers + Stripe mock*

Infrastructure files

- .github/workflows/
  - ci.yml — *Run tests on push/PR — Postgres, Redis, RabbitMQ as services*
  - deploy.yml — *Deploy to Railway on push to main*
- load-test.js — *k6 load test — 1000 VU ramping scenario, double-booking counter*

Frontend — Next.js 14 (TypeScript)

Root: `ticketing-platform/frontend/`

Root configuration files

- `package.json` — *All npm dependencies*
- `tsconfig.json` — *TypeScript config — strict mode enabled*
- `next.config.ts` — *Next.js config — API proxy rewrites for local dev*
- `tailwind.config.ts` — *Tailwind config — custom colors, fonts*
- `.env.local` — *NEXT_PUBLIC_API_URL, NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY (in .gitignore)*
- `ARCHITECTURE.md` — *Frontend architecture notes (written on Day 4)*

src/app/ — Page routes (App Router)

- `layout.tsx` — *RootLayout: html, body, Navbar, Footer, QueryProvider, AuthProvider*
- `page.tsx` — *Home: hero, category filter, events grid*
- `loading.tsx` — *Home page skeleton*
- `events/[id]/`
  - `page.tsx` — *Event Detail — tier selector, quantity picker, Add to Cart*
  - `loading.tsx`
- `cart/`
  - `page.tsx` — *Cart — items, countdown timer, order total, Proceed to Checkout*
  - `loading.tsx`
- `checkout/`
  - `page.tsx` — *Checkout — order summary + attendee info form → Stripe redirect*
- `bookings/[id]/confirmation/`
  - `page.tsx` — *Confirmation — success animation, QR code cards, Download PDF*
- `dashboard/`
  - `page.tsx` — *User Dashboard — profile, stats, bookings (auth guard)*
  - `loading.tsx`
- `organizer/events/`
  - `page.tsx` — *Organizer Dashboard — my events table, analytics (ORGANIZER guard)*
  - `new/`
    - `page.tsx` — *Create Event — multi-step form (ORGANIZER guard)*
- `auth/login/`
  - `page.tsx` — *Login form*
- `auth/register/`
  - `page.tsx` — *Register form*
- `api/auth/[...nextauth]/`
  - `route.ts` — *NextAuth or custom JWT route handler*

src/components/ — UI components

- layout/
  - Navbar.tsx
  - Footer.tsx
  - PageContainer.tsx
- events/
  - EventCard.tsx
  - EventGrid.tsx
  - CategoryFilter.tsx
  - EventSearch.tsx
  - TicketTierCard.tsx
  - TicketQuantitySelector.tsx
- booking/
  - CartItem.tsx
  - CountdownTimer.tsx
  - PriceBreakdown.tsx
  - QRCodeDisplay.tsx
  - BookingStatusBadge.tsx
- organizer/
  - CreateEventForm.tsx
  - EventAnalyticsCard.tsx
  - AttendeeTable.tsx
- ui/
  - Button.tsx
  - Input.tsx
  - Modal.tsx
  - Badge.tsx
  - Skeleton.tsx
  - Toast.tsx
  - EmptyState.tsx
- providers/
  - AuthProvider.tsx
  - CartProvider.tsx
  - QueryProvider.tsx

src/lib/ — Core utilities

- api.ts — *Axios instance — base URL, request interceptor (JWT), response interceptor (unwrap + 401 handler)*
- utils.ts — *Shared helpers: formatCurrency(), formatDate(), cn() (Tailwind class merge)*

src/hooks/ — React Query hooks

- useEvents.ts — *useQuery — events list with filter caching*
- useEvent.ts — *useQuery — single event detail*

- `useReserveTickets.ts` — *useMutation — POST /api/bookings/reserve*
- `useCreateCheckoutSession.ts` — *useMutation — POST /api/bookings/{id}/checkout*
- `useBooking.ts` — *useQuery — booking + tickets + QR codes*
- `useUserBookings.ts` — *useQuery — current user booking history*
- `useLogin.ts` — *useMutation — POST /api/auth/login*
- `useCreateEvent.ts` — *useMutation — POST /api/events*

src/stores/ — Zustand stores

- `authStore.ts` — *user, token, login(), logout(), isOrganizer() — persisted to localStorage*
- `cartStore.ts` — *currentBookingId, reservationExpiresAt, setReservation(), clearCart()*

src/types/ — TypeScript interfaces

- `api.ts` — *ApiResponse<T>, PaginatedResponse<T>*
- `event.ts` — *Event, EventSummary, Category, Venue, TicketTier, EventStatus, EventFilter*
- `booking.ts` — *Booking, Ticket, BookingState, ReserveTicketRequest, BookingResponse*
- `payment.ts` — *Payment, Refund, CheckoutSessionResponse, PaymentStatus*
- `user.ts` — *User, Role, LoginRequest, RegisterRequest*

Project File Count Summary

Total files to be created across the full Phase 1A project:

Category	File Count	Notes
Backend Java classes (main)	**~60**	Controllers, services, repositories, models, DTOs, config, security, util
Backend test classes	**~13**	Unit + integration + concurrency tests
Flyway SQL migrations	**9**	V1 through V9
Backend config files	**3**	application.yml, application-local.yml, application-test.yml
Docker / CI files	**4**	docker-compose.yml, Dockerfile, ci.yml, deploy.yml
Frontend pages (app/)	**~14**	7 route folders × ~2 files each (page.tsx + loading.tsx)
Frontend components	**~25**	layout/ + events/ + booking/ + organizer/ + ui/ + providers/
Frontend hooks, stores, types, lib	**~18**	src/hooks/, stores/, types/, lib/
Documentation + misc	**3**	README.md, ARCHITECTURE.md, postman_collection.json
TOTAL	**~149**	All files required for Phase 1A completion

Phase 1A • Sections 3, 4, 5 & Full Project Structure • April 4–24, 2026 | See Section 2 for the day-by-day execution map